

Introduction — Construire un système de gestion de boutique de zéro (avec l'IA, de la bonne façon)

Ce qu'on construit

Tout au long de cette série, on construit un système de gestion de boutique complet, de niveau professionnel, en partant d'un dossier vide jusqu'à une application fonctionnelle : plusieurs agences, chacune avec son propre prix et son propre stock par produit, des transferts de stock entre agences, des clients de passage ou enregistrés, un flux de vente/reçu complet avec génération de PDF, un accès basé sur les rôles pour les Admins, Responsables d'agence et Caissiers, des outils d'import/export en masse, et un tableau de bord en direct. Stack : Python, Django, Tailwind CSS (build Node), Chart.js.

Ce n'est pas une simple démo CRUD. Chaque élément reflète des décisions que tu prendrais réellement sur un vrai projet client ou un vrai poste, et chaque épisode part d'un plan (un diagramme UML ou une spécification écrite) avant qu'une seule ligne de code ne soit écrite.

Ce que tu seras capable de faire à la fin

À la fin de cette série, tu seras capable de prendre une idée business, de la modéliser correctement avec UML, de traduire ce modèle en projet Django avec des relations et contraintes correctes, et de livrer une application responsive, consciente des rôles, mise en cache et testée — tout en utilisant un assistant de codage IA pour aller plus vite sans jamais perdre la compréhension ni le contrôle de ce qui est construit.

Les concepts que tu vas apprendre

Conception & planification logicielle Diagrammes UML de cas d'utilisation (acteurs, limites du système, cas d'utilisation), diagrammes de classes UML (entités, relations, héritage, cardinalités), traduction d'un diagramme en modèle de données Django avant même de toucher au code, pattern de conception singleton, séparation de la logique métier en couche de service hors des vues.

Modélisation Django & ORM Conception de modèles avec ForeignKey et contraintes d'unicité, prix et stock par agence sans multi-tenancy complet, signals Django pour les effets de bord (déduction de stock, recalcul des totaux, invalidation de cache), `select_related` vs `prefetch_related` et quand utiliser chacun, isolation des querysets par rôle et par agence, migrations.

Authentification & permissions Profils utilisateur personnalisés construits sur le modèle utilisateur de Django, contrôle d'accès basé sur les rôles (Admin / Responsable d'agence / Caissier), application des permissions à la fois dans les vues et dans les querysets.

Interface d'administration Django Enregistrement des modèles, filtres de liste et recherche, éditeurs en ligne, champs calculés en lecture seule, restriction d'un modèle à une seule instance.

Templates & contexte global Context processors, héritage de templates, injection de données globales (logo de l'entreprise, paramètres) dans chaque page sans les passer manuellement.

Performance Stratégie de mise en cache et invalidation, pagination pour les grandes listes, éviter les problèmes N+1.

Traitement de données en masse Import CSV/Excel avec validation ligne par ligne et rapport d'erreurs, patterns de création/mise à jour en masse, export de données.

Modélisation du domaine métier Fige du prix/stock au moment de la vente, alertes de stock bas, cycle de vie d'une commande, numérotation des reçus, clients optionnels créés à la volée pendant une vente.

Génération de documents & PDF Génération de PDF côté serveur, templates HTML imprimables distincts des templates d'écran.

Frontend & interface utilisateur Tailwind CSS via un pipeline de build Node, conception responsive pour une vue POS mobile et un tableau de bord desktop, visualisation de données avec Chart.js.

Tests & mise en production Fonctions de service testables et isolées, notions de base de déploiement.

La compétence la plus importante : coder avec l'IA, de façon professionnelle

La chose la plus importante que cette série enseigne, ce n'est pas Django ou Tailwind — c'est comment utiliser un assistant de codage IA comme un outil professionnel sans jamais perdre le contrôle de son propre code. Cela implique une discipline précise, suivie à chaque épisode :

C'est toi qui conçois, l'IA qui assiste — jamais l'inverse. Chaque fonctionnalité part d'un plan que tu fais toi-même : un diagramme UML, une spécification écrite, une description claire du modèle et du comportement voulu. On ne demande jamais à l'IA d'inventer l'architecture. On lui donne tes décisions et on lui demande d'aider à les implémenter.

Tu lis chaque ligne avant qu'elle n'entre dans le projet. Rien n'est accepté juste parce que ça fonctionne. Le code généré est lu, compris, et — si quelque chose semble étrange, surdimensionné, ou incohérent avec le reste du code — réécrit ou rejeté. Si tu ne peux pas expliquer ce que fait un bloc de code, il n'entre pas dans le projet.

L'IA sert de levier sur les parties prévisibles, pas de jugement sur les parties importantes. Le code répétitif, le CRUD basique, l'enregistrement dans l'admin, le squelette des formulaires — excellente utilisation de l'IA. La logique critique pour le métier (déduction de stock, tarification, permissions, numérotation des reçus) est écrite délibérément et vérifiée avec soin, l'IA servant de second regard, pas de décideur.

Du contexte en entrée, de la qualité en sortie. Les prompts font référence à la spécification et aux diagrammes réels construits plus tôt dans la série, pas à des demandes vagues. Une entrée précise est ce qui garde la sortie utilisable plutôt que générique.

Discipline de contrôle de version. Des commits petits et fiables, une fonctionnalité à la fois, pour que tout changement assisté par IA soit facile à inspecter, à comparer (diff) et à annuler si nécessaire — exactement comme on travaillerait avec un collaborateur humain qu'on continue de vérifier.

La sécurité et la gestion des données restent une responsabilité humaine. Les secrets, les permissions, et tout ce qui touche à de vraies données utilisateur sont revus manuellement, à chaque fois, peu importe la façon dont le code a été produit.

Résultat : tu termines cette série non seulement avec une application de gestion de boutique fonctionnelle, mais avec une méthode de travail professionnelle et reproductible pour construire de vrais logiciels plus vite avec l'IA — sans jamais renoncer à comprendre ce que tu livres.

Résumé

Cette série montre comment construire **un système professionnel de gestion de boutique** avec **Python, Django, Tailwind CSS et Chart.js**, en partant de zéro. L'application inclut la gestion de plusieurs agences, des stocks et prix par agence, des ventes, des transferts de stock, des clients, des reçus PDF, des rôles utilisateurs (Admin, Responsable, Caissier), ainsi que des outils d'import/export et un tableau de bord.

Ce que vous apprendrez

- **Analyse et conception logicielle** avec les diagrammes UML (cas d'utilisation et classes).
- **Modélisation Django** (ORM, relations, migrations, contraintes, services, signaux).
- **Authentification et gestion des rôles.**
- **Personnalisation de l'administration Django.**
- **Templates, context processors et interface responsive** avec Tailwind CSS.
- **Optimisation des performances** (cache, pagination, optimisation des requêtes).
- **Import/export de données** (CSV/Excel).
- **Logique métier** (gestion des stocks, ventes, reçus, clients).
- **Génération de PDF.**
- **Visualisation des données** avec Chart.js.
- **Tests et déploiement.**

L'approche avec l'IA

Le principal objectif de la série est d'apprendre à utiliser l'IA **comme un assistant de développement**, et non comme un remplaçant du développeur. Les principes sont :

- Concevoir l'architecture et les modèles avant d'écrire du code.
- Utiliser l'IA pour accélérer les tâches répétitives, mais garder le contrôle des décisions importantes.
- Lire, comprendre et valider chaque ligne de code générée.
- Rédiger des prompts précis basés sur des spécifications et des diagrammes UML.
- Maintenir une bonne discipline Git avec de petits commits.
- Vérifier manuellement les aspects liés à la sécurité, aux permissions et aux données sensibles.

Objectif final

À la fin de la série, vous serez capable de **concevoir, développer, tester et déployer une application Django professionnelle**, tout en utilisant l'IA de manière responsable pour gagner en productivité sans perdre la maîtrise de votre code.